

**LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK
EKSPANSI PRODUK KEUANGAN (INHERITANCE)**

Disusun Oleh:

Muhammad Fharel

2511531010

Kelas Praktikum B

Tanggal Praktikum: 28 September 2026

Dosen Pengampu:

Nurfiah, S.ST, M.Kom

Asisten Praktikum:

Ezza Addini



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

A. Langkah Kerja Praktikum

Praktikum Modul 4 ini bertujuan untuk memahami dan mengimplementasikan konsep Code Reusability melalui Pewarisan (Inheritance), membedakan Superclass dan Subclass, serta menerapkan Upcasting dan Downcasting pada koleksi objek Java.

1. Refactoring Class Rekening (Rekening.java)

Class Rekening disesuaikan agar berfungsi sebagai kelas induk (Superclass) dengan mengubah modifier akses atribut saldo dan riwayatTransaksi menjadi protected agar dapat diakses langsung oleh subclass tanpa membuka akses ke publik.

```
package pekan4_2511531010;

import java.util.ArrayList;

public class Rekening {
    // 1. Mengunci atribut menjadi private
    private String nomorRekening;
    private String namaPemilik;
    private String pin; // Data sensitif!

    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN
    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di Constructor
        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan
PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat.");
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() {
        return nomorRekening;
    }
}
```

```

    }
    public String getNamaPemilik() {
        return namaPemilik;
    }

    // Method Otentikasi Internal
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            // Merekam riwayat (Pembuatan objek Transaksi di dalam
method)
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari
0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal nomonal
penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak
mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public void cetakMutasi() {
        System.out.println("=== RIWAYAT MUTASI REKENING ===");
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            for (Transaksi t : riwayatTransaksi) {
                t.cetakDetail();
            }
        }
    }

```

```

    }
    System.out.println("=====");
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}
}

```

Penjelasan Source Code Class Rekening.java:

- Access Modifier protected: Atribut saldo dan riwayatTransaksi diubah menjadi protected agar kelas anak (RekeningTabungan dan RekeningGiro) dapat melakukan manipulasi saldo dan pencatatan transaksi secara langsung.
- Atribut private: Atribut nomorRekening, namaPemilik, dan pin tetap dipertahankan private untuk menjamin keamanan integritas data sensitif.

2. Membuat Subclass RekeningTabungan (RekeningTabungan.java)

Class RekeningTabungan mewarisi Rekening dengan menambahkan atribut spesifik sukuBunga serta method pencatatan bunga bulanan.

```

package pekan4_2511531010;

public class RekeningTabungan extends Rekening {
    private double sukuBunga;

    public RekeningTabungan(String nomor, String nama, double saldoAwal,
String pinAwal, double sukuBunga) {
        // Konstruktor superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan:
Rp" + nominalBunga);
    }
}

```

Penjelasan Source Code Class RekeningTabungan.java:

- Keyword `extends` & `super()`: Menggunakan `extends` Rekening untuk menandakan hubungan Is-A. Kata kunci `super()` dipanggil di baris pertama konstruktor untuk menginisialisasi atribut pada superclass.
- Method `tambahBungaAkhirBulan()`: Menghitung akumulasi bunga berdasarkan persentase `sukuBunga`, menambahkan nilai bunga ke saldo, serta merekamnya ke `riwayatTransaksi`.

3. Membuat Subclass RekeningGiro (RekeningGiro.java)

Class `RekeningGiro` mewarisi `Rekening` dengan fasilitas pinjaman (overdraft) khusus untuk kebutuhan pebisnis.

```
package pekan4_2511531010;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}
```

Penjelasan Source Code Class RekeningGiro.java:

- Atribut `batasOverdraft`: Menyimpan limit batas pinjaman transaksi yang diizinkan untuk akun Giro.

4. Class Transaksi.java

Jangan lupa untuk membuat class `Transaksi` sama seperti praktikum sebelumnya.

```
package pekan4_2511531010;

public class Transaksi {

    private String idTransaksi;
    private String jenis;
    private double nominal;
```

```
// Constructor
public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}

public String getIdTransaksi() {
    return idTransaksi;
}
public String getJenis() {
    return jenis;
}
public double getNominal() {
    return nominal;
}

public void cetakDetail() {
    System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
" | Nominal: Rp" + nominal);
}
}
```

Penjelasan Source Code Class Transaksi.java:

- Access Modifier private pada Atribut (idTransaksi, jenis, nominal)
Semua atribut dikunci menggunakan modifier private agar tidak dapat dimanipulasi secara langsung dari luar class.
- Implementasi Method Getter
Menyediakan method getIdTransaksi(), getJenis(), dan getNominal() untuk membaca nilai atribut secara aman tanpa memberikan akses untuk mengubahnya (Read-Only).
- Method Setter
Setter sengaja tidak dibuat karena objek transaksi dirancang bersifat immutable (tidak boleh diubah nilainya setelah dibuat).

B. Latihan Praktikum

Soal:

Sekarang sistem telah memiliki berbagai bentuk rekening. Silahkan memperbarui antarmuka CLI di **Main.java** agar mendukung produk-produk baru ini secara interaktif.

Tugas 1: Pembuatan Multi-Produk (40%)

Modifikasi case 1: **Buka Rekening Baru** di Main.java.

1. Setelah user memasukkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
2. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
3. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
4. Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

Tugas 2: Simulasi Waktu Akhir Bulan : 60%

1. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
2. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator *instanceof*.
3. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
4. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

Kode Program Main.java yang lengkap:

```
package pekan4_2511531010;

import java.util.Scanner;
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;

        while (isRunning) {
            System.out.println("\n=== SISTEM PERBANKAN MINI ===");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("6. Cetak Mutasi (Riwayat)");
            System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            int pilihan = scanner.nextInt();
            scanner.nextLine();

            switch (pilihan) {
```

```

        case 1:
            System.out.print("Masukkan No Rekening: ");
            String no = scanner.nextLine();
            System.out.print("Masukkan Nama Pemilik: ");
            String nama = scanner.nextLine();
            System.out.print("Masukkan Saldo Awal: ");
            double saldoAwal = scanner.nextDouble();
            scanner.nextLine();
            System.out.print("Masukkan PIN (6 digit): ");
            String pin = scanner.nextLine();

            System.out.println("Pilih Produk: 1. Tabungan Umum | 2.
Giro Bisnis");

            System.out.print("Pilihan: ");
            int jenisProduk = scanner.nextInt();
            scanner.nextLine();

            if (jenisProduk == 1) {
                System.out.print("Masukkan Suku Bunga (%): ");
                double sukuBunga = scanner.nextDouble();
                scanner.nextLine();
                akunAktif = new RekeningTabungan(no, nama, saldoAwal,
pin, sukuBunga);
            } else if (jenisProduk == 2) {
                System.out.print("Masukkan Batas Overdraft: ");
                double overdraft = scanner.nextDouble();
                scanner.nextLine();
                akunAktif = new RekeningGiro(no, nama, saldoAwal,
pin, overdraft);
            } else {
                System.out.println("Pilihan produk tidak valid!");
                break;
            }

            daftarRekening.add(akunAktif);
            break;

        case 2:
            if (akunAktif != null) {
                System.out.print("Masukkan nominal setor: ");
                double nominalSetor = scanner.nextDouble();
                akunAktif.setorTunai(nominalSetor);
            } else {
                System.out.println("Error: Belum ada rekening yang
dibuat!");
            }
            break;

        case 3:
            if (akunAktif == null) {
                System.out.println("Belum ada rekening aktif!
Silahkan buka rekening terlebih dahulu.");
                break;
            }

```

```

        System.out.print("Masukkan PIN Anda: ");
        String pinTarik = scanner.nextLine();

        if (akunAktif.otentikasi(pinTarik)) {
            System.out.print("Masukkan nominal tarik: ");
            double nominalTarik = scanner.nextDouble();
            akunAktif.tarikTunai(nominalTarik);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
        }
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka
rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Belum ada rekening yang terdaftar
di sistem.");
        } else {
            System.out.print("Masukkan Nomor Rekening yang ingin
diaktifkan: ");

            String cariNoRek = scanner.nextLine();
            boolean ditemukan = false;

            for (Rekening r : daftarRekening) {
                if
(r.getNomorRekening().equalsIgnoreCase(cariNoRek)) {
                    akunAktif = r;
                    ditemukan = true;
                    System.out.println("Berhasil mengganti ke
akun milik: " + r.getNamaPemilik());
                    break;
                }
            }

            if (!ditemukan) {
                System.out.println("Error: Nomor rekening tidak
ditemukan!");
            }
        }
        break;

    case 6:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka
rekening! Silahkan buka rekening terlebih dahulu.");

```

```

        } else {
            System.out.print("Masukkan PIN: ");
            String pinMutasi = scanner.nextLine();

            if (akunAktif.otentikasi(pinMutasi)) {
                akunAktif.cetakMutasi();
            } else {
                System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
            }
        }
        break;

    case 7:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka
rekening!");
        } else {
            if (akunAktif instanceof RekeningTabungan) {
                RekeningTabungan tabungan = (RekeningTabungan)
akunAktif;

                tabungan.tambahBungaAkhirBulan();
            } else {
                System.out.println("Gagal: Fitur bunga akhir
bulan hanya berlaku untuk Rekening Tabungan.");
            }
        }
        break;

    case 0:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;

    default:
        System.out.println("Pilihan tidak valid!");
    }
}
scanner.close();
}
}

```

1. Tugas 1: Pembuatan Multi-Produk

```

2. case 1:
3.     System.out.print("Masukkan No Rekening: ");
4.     String no = scanner.nextLine();
5.     System.out.print("Masukkan Nama Pemilik: ");
6.     String nama = scanner.nextLine();
7.     System.out.print("Masukkan Saldo Awal: ");
8.     double saldoAwal = scanner.nextDouble();
9.     scanner.nextLine();
10.    System.out.print("Masukkan PIN (6 digit):
    ");

```

```

11.         String pin = scanner.nextLine();
12.
13.         System.out.println("Pilih Produk: 1.
Tabungan Umum | 2. Giro Bisnis");
14.         System.out.print("Pilihan: ");
15.         int jenisProduk = scanner.nextInt();
16.         scanner.nextLine();
17.
18.         if (jenisProduk == 1) {
19.             System.out.print("Masukkan Suku Bunga
(%): ");
20.             double sukuBunga = scanner.nextDouble();
21.             scanner.nextLine();
22.             akunAktif = new RekeningTabungan(no,
nama, saldoAwal, pin, sukuBunga);
23.         } else if (jenisProduk == 2) {
24.             System.out.print("Masukkan Batas
Overdraft: ");
25.             double overdraft = scanner.nextDouble();
26.             scanner.nextLine();
27.             akunAktif = new RekeningGiro(no, nama,
saldoAwal, pin, overdraft);
28.         } else {
29.             System.out.println("Pilihan produk tidak
valid!");
30.             break;
31.         }
32.
33.         daftarRekening.add(akunAktif);
34.         break;

```

Penjelasan Kode Tugas 1:

- Program menerima masukan data rekening utama (Nomor, Nama, Saldo Awal, PIN) lalu memberikan pilihan tipe produk yang ingin dibuka.
- Jika pengguna memilih opsi 1 (Tabungan), sistem meminta input persentase sukuBunga dan menginstansiasi objek RekeningTabungan.
- Jika pengguna memilih opsi 2 (Giro), sistem meminta input limit batasOverdraft dan menginstansiasi objek RekeningGiro.
- Objek subclass yang terbentuk disimpan ke dalam ArrayList<Rekening> daftarRekening melalui mekanisme Upcasting secara implisit (subclass otomatis dikenali sebagai tipe superclass-nya).

Contoh Output Tugas 1:

```
=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123456789
Masukkan Nama Pemilik: Jennie
Masukkan Saldo Awal: 500000
Masukkan PIN (6 digit): 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 2
Rekening atas nama Jennie berhasil dibuat.
```

```
=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 987654321
Masukkan Nama Pemilik: Fharel
Masukkan Saldo Awal: 5000000
Masukkan PIN (6 digit): 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 2000000
Rekening atas nama Fharel berhasil dibuat.
```

Penjelasan Output Tugas 1:

Menampilkan dua skenario pendaftaran rekening baru menggunakan pilihan produk yang berbeda pada Sistem Perbankan Mini. Pada Versi 1, pengguna memilih opsi 1 (Tabungan Umum), di mana sistem meminta parameter tambahan berupa suku bunga 2% dan berhasil menginstansiasi objek RekeningTabungan. Sementara pada Versi 2, pengguna memilih opsi 2 (Giro Bisnis), di mana sistem meminta input batas overdraft Rp2.000.000 dan berhasil menginstansiasi objek RekeningGiro. Kedua

objek subclass tersebut secara otomatis disimpan ke dalam variabel referensi superclass daftarRekening memanfaatkan konsep Upcasting.

2. Tugas 2: Simulasi Waktu Akhir Bulan

```
case 7:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        if (akunAktif instanceof RekeningTabungan) {
            RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
            tabungan.tambahBungaAkhirBulan();
        } else {
            System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
        }
    }
    break;
```

Penjelasan Kode Tugas 2:

- Pengecekan Tipe Objek (instanceof): Sebelum mengeksekusi penambahan bunga, sistem memeriksa apakah akunAktif saat ini merupakan instansi dari kelas RekeningTabungan.
- Jika hasil dari instanceof bernilai true, dilakukan konversi tipe downcasting (RekeningTabungan) akunAktif agar method khusus tambahBungaAkhirBulan() dapat dipanggil tanpa memicu error saat runtime.
- Penolakan Hak Akses: Jika akunAktif bukan RekeningTabungan (misalnya RekeningGiro), transaksi dibatalkan dan sistem memunculkan pesan penolakan sesuai ketentuan modul.

Contoh Output Tugas 2:

- a. Pada Akun Tabungan (Berhasil)

```

=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123456789
Masukkan Nama Pemilik: Jennie
Masukkan Saldo Awal: 1000000
Masukkan PIN (6 digit): 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 2
Rekening atas nama Jennie berhasil dibuat.

=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Bunga 2.0% berhasil ditambahkan: Rp20000.0

```

b. Pada Akun Giro (Gagal / Ditolak)

```

=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 987654321
Masukkan Nama Pemilik: Fharel
Masukkan Saldo Awal: 5000000
Masukkan PIN (6 digit): 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 2000000
Rekening atas nama Fharel berhasil dibuat.

=== SISTEM PERBANKAN MINI ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

```

Penjelasan Output Tugas 2:

Pada skenario pertama, akun yang aktif adalah RekeningTabungan, sehingga pemeriksaan instanceof bernilai true dan perhitungan bunga akhir bulan sebesar Rp20.000 sukses dijalankan. Sebaliknya pada skenario kedua, saat akun aktif berupa RekeningGiro, sistem berhasil mendeteksi perbedaan tipe objek dan menolak eksekusi bunga dengan menampilkan pesan peringatan.

C. Link Repository GitHub

https://github.com/MuhammadFharel/PrakPBO_B_2511531010